

# A Recursive DNS Resolver on AF\_XDP

*Kernel-bypass packet I/O, and what it costs*

Divisht (24110112)<sup>1</sup> Ankit Saha (24110042)<sup>1</sup> Jahan Zaib (24110140)<sup>1</sup>  
Mateen Sharief (24110197)<sup>1</sup> Aditya Kumar (22110015)<sup>1</sup> Adil Khan N (23110013)<sup>1</sup>

<sup>1</sup>Indian Institute of Technology Gandhinagar

---

**Abstract.** We present a recursive DNS resolver whose packet path bypasses the Linux network stack using an AF\_XDP socket, together with a conventional UDP server built as a controlled baseline. Both servers link the same compiled resolver object, so the comparison isolates the transport as a single independent variable. The report first establishes the baseline architecture—the XDP classifier, the UMEM and ring machinery, and the concurrency model that makes a multi-threaded resolver safe on single-producer rings—and then describes four extensions: a counting Bloom filter that allows a cache miss to avoid the cache mutex entirely, a set of DNS response acceptance rules that defend against cache poisoning, multi-queue operation across CPU cores, and an adversarial test harness. The Bloom filter reduces cold-path cache lookups from 168 ns to 121 ns single-threaded (1.38×), widening to a 3.79× advantage under eight-thread contention as the skipped mutex becomes fully contested—confirming that the benefit grows with the contention the filter was designed to eliminate. All seven implemented poisoning attacks are defeated. 329 automated checks pass across four suites under AddressSanitizer, UndefinedBehaviorSanitizer and ThreadSanitizer.

**Keywords:** DNS, AF\_XDP, eBPF, XDP, kernel bypass, cache poisoning, Bloom filter.

---

## 1 Problem Statement and Objectives

A DNS query is a small datagram—typically under 100 bytes—that demands a reply within milliseconds. The Linux network stack is not designed for this shape of workload. Every received frame is wrapped in a `sk_buff`, traverses the IP and UDP layers, and is then copied once more across the kernel/user boundary before the resolver sees a single byte. For bulk transfers that per-packet cost is amortised over a large payload; for DNS it is the dominant term.

AF\_XDP is a socket family that allows a user-space process to receive frames directly from a NIC driver, into memory it registered itself, with the kernel's protocol stack skipped entirely. The trade is explicit: the application must now do the work the stack was doing, and must do it correctly.

### 1.1 Objectives

- O1.** Build a working recursive DNS resolver whose data path uses an AF\_XDP socket: an XDP program redirects UDP/53 traffic to the socket, and user space crafts raw Ethernet/IPv4/UDP/DNS frames for transmission.
- O2.** Build a conventional UDP server as a control, structured so that the *only* difference between the two is the transport.
- O3.** Quantify what the bypass buys in latency, through-

put and CPU cost, and what it costs in implementation complexity.

- O4.** Extend the baseline with features that a resolver actually needs—caching, response validation, multi-core scaling—and evaluate each on its own terms.

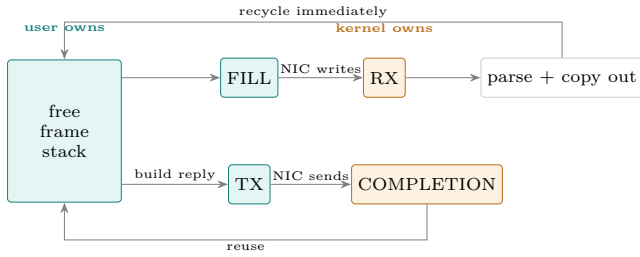
### 1.2 Scope and non-goals

The resolver performs full iterative resolution from the root servers. It does not implement DNSSEC validation, and we do not claim cryptographic authentication of responses anywhere in this report. Section 4.2 describes defences that raise the cost of off-path spoofing; they are not a substitute for DNSSEC and we are explicit about that distinction.

## 2 Background

### 2.1 Why the stack is expensive for small packets

On ingress, the driver DMAs a frame into memory and raises an interrupt. The kernel allocates an `sk_buff` descriptor, walks the protocol layers, matches a socket, and finally copies the payload into the user's buffer. Two costs dominate for small packets: the descriptor allocation and cache pressure it creates, and the user-space copy, which cannot be elided because the kernel/user boundary must be preserved.



**Figure 1:** UMEM frame lifecycle. Recycling the receive frame *before* resolution begins bounds pool occupancy by batch size rather than by resolver latency (§3.3).

## 2.2 XDP and eBPF

eBPF allows verified, sandboxed programs to run at fixed hook points inside the kernel. XDP is the earliest such hook on the receive path, running in the driver before an `sk_buff` exists. An XDP program inspects the raw frame and returns a verdict: `XDP_PASS` (continue into the stack), `XDP_DROP`, `XDP_TX`, or `XDP_REDIRECT`.

## 2.3 AF\_XDP

XDP\_REDIRECT into an `XSKMAP` delivers the frame to an AF\_XDP socket. The socket is backed by a **UMEM**: a contiguous region of user memory, registered with the kernel and divided into fixed-size frames. Four single-producer/single-consumer rings carry descriptors—not data—between kernel and user space:

- **FILL**: user → kernel.
- **RX**: kernel → user.
- **TX**: user → kernel.
- **COMPLETION**: kernel → user.

A frame’s ownership cycles `FILL` → `RX` → (application) → `TX` → `COMPLETION` → `FILL`. Managing that cycle correctly is the bulk of the user-space work.

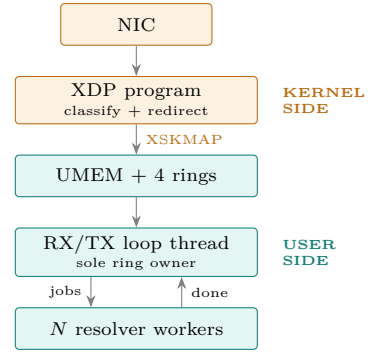
One distinction matters throughout: the XDP *attach mode* (native vs. generic) is independent of the AF\_XDP *bind flag* (`XDP_ZEROCOPY` vs. `XDP_COPY`). Our implementation prints the negotiated bind flag at startup, and that is the only figure we cite when discussing zero-copy.

# 3 Baseline Architecture

The system splits cleanly along the kernel/user boundary (Figure 2).

## 3.1 Kernel side: classification

The XDP program does considerably more than match a destination port. A frame is redirected only if it is IPv4, unfragmented, UDP, destined for port 53, carries a complete 12-byte DNS header, has `QR=0` (a query), and declares `QDCOUNT=1`. Anything else returns `XDP_PASS`.



**Figure 2:** Data path. Workers never touch a ring.

```
if (bpf_map_lookup_elem(&xsk_map, &qidx))
    return bpf_redirect_map(&xsk_map, qidx,
                           XDP_PASS);
return XDP_PASS;
```

Three properties of this design are deliberate. The **redirect key is the RX queue index**, so multi-queue operation (Section 4.3) required no BPF change. The **fallback is XDP\_PASS**: if no socket is registered the machine degrades to an ordinary host rather than black-holing port 53. There is **no bpf\_printk on the hot path**; counters live in a `BPF_MAP_TYPE_PERCPU_ARRAY`.

User space re-validates every length after receiving a frame. The kernel filter is a fast path, not a trust boundary.

## 3.2 User side: the concurrency model

AF\_XDP rings are single-producer/single-consumer lock-free structures. The natural architecture—each resolver thread transmits its own answer—places  $N$  producers on the TX ring. It compiles, runs, and passes a `dig` test. It is also incorrect. The reserve operation in `libxdp` is:

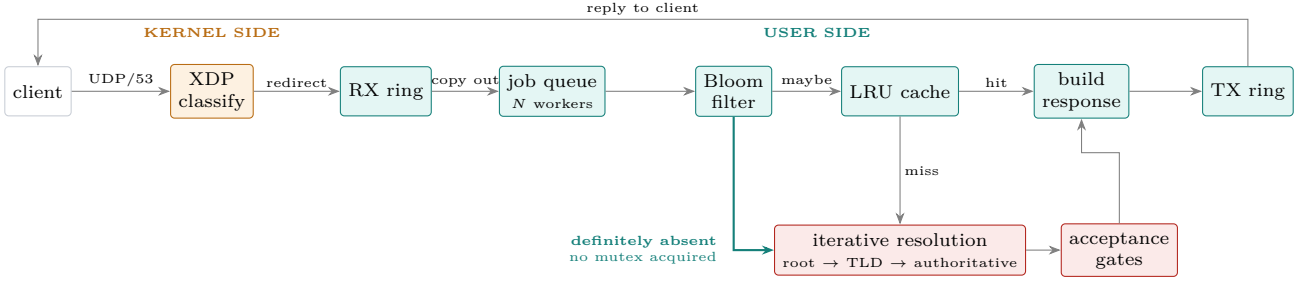
```
*idx = prod->cached_prod;
prod->cached_prod += nb; /* non-atomic RMW */
```

Two threads reserving concurrently obtain the *same* index. The failure manifests as elevated packet loss under load, which reads as “AF\_XDP is lossy” rather than as a data race in the application.

Our design enforces a strict invariant: *exactly one thread touches a socket’s four rings and its frame allocator*. Resolver workers compute bytes and nothing else. Completed responses return to the loop thread through a mutex-protected queue plus an `eventfd`.

## 3.3 User side: frame lifetime

An iterative resolution from root takes on the order of a second. We therefore copy the DNS payload ( $\leq 512$  bytes) into a self-contained job structure and return the frame to the FILL ring in the same batch it arrived in, bounding pool occupancy by batch size rather than by resolver latency. The cost is one 512-byte `memcpy` per query.



**Figure 3:** End-to-end query workflow. A “definitely absent” verdict skips the cache mutex entirely and goes straight to resolution, which is the common case for a resolver serving cold traffic.

|        | Real backend | Stub backend     |
|--------|--------------|------------------|
| AF_XDP | af_xdp_user  | af_xdp_user_stub |
| UDP    | server       | server_stub      |

**Table 1:** Four binaries, two pairs. Within each row, both binaries link the *same* compiled backend object.

### 3.4 Resolver core

Independent of transport, the resolver implements wire-format encoding/decoding with bounded decompression; iterative resolution from root hints with referral following, glue handling, and CNAME chasing; a thread-safe LRU cache keyed on (name, qtype) with real TTL accounting; EDNS0 with a 1232-byte payload ceiling; and a TCP/53 listener for truncated responses.

### 3.5 The control arm and single-variable methodology

We build four binaries (Table 1). Within each row, both link the same `dns_backend.o` or `resolver_stub.o` — not a copy, but the identical object file called through the identical function declared in `dns_iface.h`. This reduces the experiment to one independent variable: the transport.

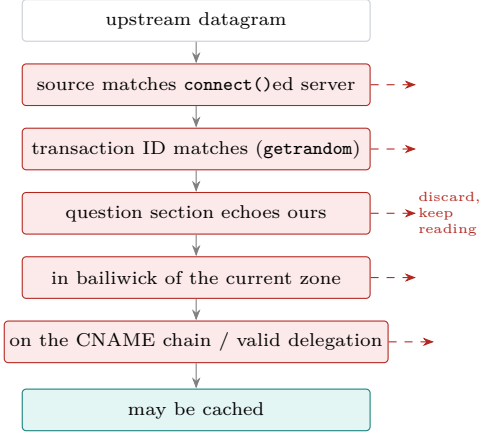
## 4 Extensions

### 4.1 A counting Bloom filter in front of the cache

**Motivation.** A cache lookup acquires a mutex, hashes the name, and walks a bucket chain. On a miss, all of that work is wasted. A Bloom filter can answer “definitely not cached” without touching the map at all. The goal is not a faster hit—a hit still walks the bucket. The goal is that a miss never takes the lock.

**Why a plain Bloom filter is wrong here.** This cache deletes: entries leave by LRU eviction and TTL expiry. A plain filter cannot clear a bit without risking a false negative for some other key that shares it. Bits would only ever accumulate, the false-positive rate would climb toward 1, and the filter would decay into a branch that always answers “maybe.”

**Design.** We use a counting Bloom filter with  $m = 65536$  one-byte counters (64 KiB),  $k = 4$  hash functions



**Figure 4:** Response acceptance pipeline. Discarding a datagram does not end the read: the resolver continues until its timeout, preventing a single spoofed packet from denying service.

derived from a single 64-bit FNV-1a hash by the Kirsch–Mitzenmacher construction  $g_i(x) = h_1(x) + i \cdot h_2(x)$ , with  $h_2$  forced odd so the  $k$  indices are always distinct. Deletion decrements; counters saturate at 255 rather than wrapping.

At cache capacity  $n = 4096$ ,  $m/n = 16$  and the predicted false positive rate is

$$\left(1 - e^{-kn/m}\right)^k \approx 0.24\%.$$

The measured rate over  $10^5$  probes is 0.255%.

**Concurrency.** Reads are lock-free with relaxed atomics; updates ride the existing cache mutex. Both race directions are benign. A lost hit sends the resolver to resolution a second time; a stale non-zero after deletion takes the lock, finds nothing, and treats it as an ordinary miss.

### 4.2 Response acceptance rules

A resolver that caches whatever it is told is a cache-poisoning appliance. We added three structural rules and two response-matching rules to the baseline.

#### Bailiwick

A server may only speak about names at or below the zone we are currently querying it about. The predicate `dns_name_in_bailiwick` gates referrals, glue

and answers alike, checking at a label boundary so that `notexample.com` is not accepted as a match for `example.com`.

### CNAME chain membership

Answers are accepted only for names on the CNAME chain beginning at the queried name. This catches an attack bailiwick alone does not: a record injected for a different name *inside the server’s own zone*. The answer section is rescanned until the chain stops growing so that out-of-order records are not dropped.

### Referral progress

A delegation must be strictly *below* the current zone so a server cannot refer us to itself indefinitely.

### Transaction identifiers

Transaction IDs are drawn from `getrandom()` in batches of 64. There is deliberately no weak fallback: if the CSPRNG is unavailable the query fails rather than degrading to a guessable LCG.

### Question-section matching and read persistence

A response must echo the question: QDCOUNT, QNAME (case-insensitively), QTYPE and QCLASS. After rejecting a spoofed datagram the resolver continues reading until the timeout, so an attacker cannot convert a failed spoof into a denial of service.

## 4.3 Multi-queue operation

An AF\_XDP socket binds one RX queue. Scaling across cores is two separate problems. On the **kernel side**, RSS must spread flows across queues and each queue’s interrupt must be pinned to its own core—both are properties of the device the XDP program itself required no change, since it already redirects on `ctx->rx_queue_index`. On the **user side**, each RX queue receives its own `xsk_env`: socket, UMEM, rings, frame pool, and one pinned thread. Nothing is shared between loops, preserving the single-writer invariant. The pending-job queue is shared; the completion queue is not, because a reply must depart through the socket it arrived on.

## 4.4 An adversarial test hierarchy

We implemented a second DNS hierarchy that lies. It mounts a root, TLD and authoritative server on loopback aliases and mounts seven distinct attacks (Table 3). Every attack is paired with a follow-up query against the zone it attempted to hijack.

# 5 Evaluation

## 5.1 Bloom filter

Table 2 gives the measured cost across thread counts. Single-threaded, the filter reduces the cold-path lookup

| Threads | Bloom<br>(ns/op) | No Bloom<br>(ns/op) | Speedup |
|---------|------------------|---------------------|---------|
| 1       | <b>121</b>       | 168                 | 1.38×   |
| 2       | <b>285</b>       | 794                 | 2.78×   |
| 4       | <b>491</b>       | 1522                | 3.10×   |
| 8       | <b>1068</b>      | 4051                | 3.79×   |

**Table 2:** Cache lookup cost, 90 % cold queries, 2 M iterations per thread (mean). The speedup grows with thread count because the filter skips a *contended* mutex—the effect it was designed for.

time from 168 ns to 121 ns (1.38×). More importantly, the gap widens substantially under contention: at eight threads the no-Bloom path degrades to 4051 ns per lookup against 1068 ns with the filter—a 3.79× advantage—with the speedup growing monotonically at every thread count (1.38×, 2.78×, 3.10×, 3.79×). This directly confirms the design hypothesis: the saving comes from skipping a *contended* mutex, not merely from four byte loads. A filter that saved the same amount regardless of thread count would indicate a different mechanism.

On the hit path, the overhead of four relaxed byte loads is negligible: the Bloom variant never slows the hit case, because a “maybe” verdict simply falls through to the existing cache lookup.

In 90 % cold conditions, 89.5–89.8 % of lookups bypassed the mutex entirely at every thread count tested, with false-positive rates between 0.024 % and 0.068 %—all well below the analytically predicted 0.24 %.

**Throughput perspective.** With the Bloom filter, aggregate lookup throughput stays between 7.00 and 8.22 Mlookup/s regardless of thread count, reflecting efficient parallel execution. Without it, throughput collapses from 5.93 Mlookup/s at one thread to 1.96 Mlookup/s at eight as mutex contention dominates.

## An optimisation that was a regression

The filter’s first implementation maintained three shared `_Atomic` counters incremented with `fetch_add` on every lookup—a lock-prefixed read-modify-write on a shared cache line. The structure built to avoid contended shared writes was performing one itself, for telemetry. Statistics are now sharded per thread, one cache line each, so the load/store pair compiles to plain moves.

A blocked layout confining all  $k$  slots to one cache line was also tried and measured worse (41.6 ns vs. 34.2 ns single-threaded): the deduplication required for add/delete symmetry costs more than the cache misses it saves when 64 KiB largely resides in L2.

## 5.2 Acceptance rules

All seven attacks are defeated (Table 3). Attack 4 is the one a resolver implementing only bailiwick checks would fail, and attack 7 verifies that rejection does not become denial of service—the genuine reply is still accepted after the spoof is discarded.

| # | Attack                            | Result   |
|---|-----------------------------------|----------|
| 1 | Out-of-bailiwick referral         | defeated |
| 2 | Out-of-bailiwick glue             | defeated |
| 3 | Foreign record in answer section  | defeated |
| 4 | In-bailiwick, off-chain injection | defeated |
| 5 | Correct question, wrong txid      | defeated |
| 6 | Correct txid, wrong question      | defeated |
| 7 | Spoof first, genuine reply second | defeated |

**Table 3:** Attacks mounted by the adversarial hierarchy. Each is verified both in the returned answer and in the cache afterwards.

### 5.3 Feature interaction under concurrency

Features that are individually correct may still interact badly. We drive all of them simultaneously from 16–24 threads: repeated “hot” names producing cache hits, unique “cold” names producing Bloom rejections, attack names exercising the acceptance rules, and 4000 churn names forcing LRU eviction so that Bloom counters move in both directions. A separate test races 24 threads on the first lookup of a single uncached name.

The counters recorded after such a run are mutually consistent:

|                  |      |               |      |
|------------------|------|---------------|------|
| cache hits       | 286  | bloom lookups | 2858 |
| cache misses     | 2572 | bloom rejects | 2572 |
| bloom slots used | 9519 | (of 65536)    |      |

Hits plus misses equal Bloom lookups exactly ( $286 + 2572 = 2858$ ), and rejections equal misses exactly: under adversarial concurrent load the filter and the map never disagreed. Had a counter been decremented for the wrong key, rejections would exceed misses. The occupancy of 9519 slots for approximately 2572 cold-path entries is  $\approx 3.7$  per entry against a ceiling of  $k = 4$ , consistent with hash collisions and with no counter leak.

### 5.4 Transport comparison

Table ?? remains unpopulated. Three methodological caveats apply regardless.

**The control arm is deliberately simple.** Our UDP server uses a single socket and a single `recvfrom` loop with a worker pool, without `SO_REUSEPORT` or `recvmsg`. The defensible claim is that AF\_XDP outperforms a single-threaded receive loop, not that it outperforms UDP sockets in general.

**Generic XDP cannot demonstrate bypass.** The `-S` mode hooks after `sk_buff` allocation. A useful consistency check: under `-S`, the poll and copy variants should produce near-identical figures.

**Link parity.** Both arms must use the same physical link; a comparison over loopback versus `veth` introduces a second uncontrolled variable.

### 5.5 Verification summary

Table 4 summarises what has been verified. The offline integration suite grew from 28 to 66 checks during development, expanding to cover AAAA queries,

| Check                   | Result       | Tooling                        |
|-------------------------|--------------|--------------------------------|
| Build, 4 binaries + BPF | 0 warnings   | <code>-Wall -Wextra</code>     |
| BPF verifier            | accepted     | <code>bpftool prog load</code> |
| Unit tests              | 227 / 227    | ASan + UBSan                   |
| Race detection          | 0 warnings   | ThreadSanitizer                |
| Offline integration     | 66 / 66      | fake hierarchy                 |
| Adversarial             | 36 / 36      | evil hierarchy                 |
| AF_XDP data path        | not executed | —                              |

**Table 4:** Automated verification. 329 checks in total. The AF\_XDP RX/TX path is compiled and race-reviewed but not exercised: the test environment lacked `CAP_NET_ADMIN` and a zero-copy-capable driver.

EDNS0 payload negotiation, TCP/53 with pipelining, and truncation fallback—all tested against the honest fake hierarchy with 0 failures. The final row is stated deliberately: the RX/TX data path is validated on the evaluation host with a single queue before multi-queue operation is trusted.

## 6 Testing Methodology

Testing is organised so that every layer can be exercised without privileges, a NIC, or network access.

- **Unit tests** (227 checks) cover wire-format encoding and decoding, malicious compression pointers, cache TTL and LRU semantics, and Bloom filter invariants under ASan and UBSan.
- **Race detection** runs the same suite under ThreadSanitizer as a separate build. This exists specifically because the Bloom filter’s lock-free read is invisible to a single-threaded run. Result: 0 warnings.
- **Offline integration** (66 checks) stands up an honest root/TLD/authoritative chain on loopback and exercises referral following, glue, missing-glue fallback, CNAME chains, NXDOMAIN, NODATA, AAAA queries, EDNS0 payload negotiation, TCP/53 with request pipelining, and truncated-UDP/TCP-retry fallback. All 66 pass with 0 failures in approximately two seconds.
- **Adversarial integration** (36 checks) runs the hostile hierarchy of Table 3 plus the concurrent feature-interaction tests.

A note on the Bloom filter tests: the decisive one adds a key twice, deletes it once, and asserts it still reads as present. A plain bit filter cannot express this, which is exactly why the counting variant is required.

## 7 Issues Found and Fixed

We record these because a test suite is only worth describing if it found something.

### B1. Truncation flag from uninitialised memory.

The response header is written last, but the truncation path read `out[2..3]` back before anything had written it. On a dirty buffer this set TC on

responses that fit, causing `dig` to retry over TCP intermittently. The test now poisons the output buffer with `0xFF`.

- B2. Case-sensitive hash, case-insensitive comparison.** `EXAMPLE.com` hashed to a different bucket from `example.com` and never matched. Silent, invisible unless queried with mixed case.
- B3. `memcpy` from `NULL` on negative cache entries—** detected by UBSan.
- B4. Duplicated TCP listener start.** The listener was started twice; the second bind fails, the handle is overwritten with `NULL`, and the first server runs with no means of stopping it.
- B5. Bloom statistics as a regression** (Section 4.1).
- B6. A teardown path that could close `stdin`.** Completion `eventfd` handles are zero-initialised, and 0 is a valid descriptor; the error path closed every handle it found.
- B7. A concurrency test that tested nothing.** The key was selected as `seed % N` while the branch used `seed & 1`, so insertions only touched odd keys and lookups only even ones. A hit rate of exactly 0.0% exposed it.

## 8 Comparison with a Prior Implementation

We compared our design against a publicly available `AF_XDP` DNS resolver [?].

**The kernel side agrees.** Both attach one program per interface and redirect on `ctx->rx_queue_index`, confirming that our multi-queue approach is canonical. Our program additionally validates the IP header length, fragmentation, and DNS header; records counters in a `PERCPU_ARRAY` rather than via `bpf_printk`; and falls back to `XDP_PASS` rather than `XDP_ABORTED`.

**The user side differs structurally.** In that implementation, worker threads invoke `transmit`, completion and frame-release routines concurrently on a shared socket, constituting a multi-writer race on the TX ring and frame allocator. Workers also retain the RX UMEM frame for the duration of resolution, which starves the FILL ring under concurrency.

**One respect in which it is better.** It rewrites the response into the RX frame in place, avoiding the copy of Section 3.3 entirely. That is cheaper, and it is sound provided resolution is single-threaded. Our two copies purchase a safe concurrent backend.

The accompanying documentation also interprets native attach mode as evidence that zero-copy is in effect. As discussed in Section 2.3, these are independent properties.

## 9 Challenges Faced

**Reasoning about correctness without a data path.** The development environment provided a single CPU, no `CAP_NET_ADMIN`, and no zero-copy-capable

driver. Every conclusion about ring safety had to be reached by reading the `libxdp` implementation and constructing user-space tests that exercise the same concurrency patterns.

**Distinguishing benign races from real ones.** The Bloom filter’s lock-free read is safe, but arguing so required enumerating both race directions and showing each is harmless. `ThreadSanitizer` validates the mechanics; the consistency check on rejection and miss counts validates the semantics.

**Benchmarks that measure the wrong thing.** Our first Bloom benchmark was dominated by `snprintf` in the timed loop. Our first harness compared loopback against `veth`. In both cases the numbers looked plausible and were meaningless.

**Attacks that bailiwick alone does not stop.** Recognising that a record injected within a server’s own zone but off the CNAME chain passes a bailiwick check required working through the attack model explicitly.

**Tests that pass without testing anything.** Defect B7 reported green while exercising nothing. It was caught only because the hit rate it printed was exactly zero.

## 10 Limitations and Future Work

- **No DNSSEC validation.** The rules of Section 4.2 raise the cost of off-path spoofing substantially, but they are structural checks, not authentication.
- **No upstream TCP fallback.** Downstream TCP is implemented; when an upstream server truncates its response we fall through to the next server rather than retrying over TCP.
- **NODATA responses are not cached,** so every AAAA query for an A-only host is a permanent cache miss.
- **The backend bounds throughput below the packet path.** In-flight resolutions are capped at the worker count; an event-driven upstream path would remove the bottleneck.
- **Per-packet allocation.** Job structures are `malloced` on the loop thread; a preallocated pool would remove the allocator from the packet path.
- **A stronger control arm.** Adding `SO_REUSEPORT` plus `recvmmsg` would make the transport comparison considerably more defensible.

## References

- [1] T. Høiland-Jørgensen et al., “The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel,” *Proc. CoNEXT*, 2018.
- [2] Linux kernel documentation, “AF\_XDP,” [https://docs.kernel.org/networking/af\\_xdp.html](https://docs.kernel.org/networking/af_xdp.html).
- [3] xdp-project, “XDP Hands-On Tutorial,” <https://github.com/xdp-project/xdp-tutorial>.

- [4] B. H. Bloom, "Space/Time Trade-offs in Hash Coding with Allowable Errors," *Communications of the ACM*, 13(7), 1970.
- [5] L. Fan, P. Cao, J. Almeida, A. Z. Broder, "Summary Cache: A Scalable Wide-Area Web Cache Sharing Protocol," *IEEE/ACM Trans. Networking*, 8(3), 2000.
- [6] A. Kirsch and M. Mitzenmacher, "Less Hashing, Same Performance: Building a Better Bloom Filter," *Random Structures & Algorithms*, 33(2), 2008.
- [7] P. Mockapetris, "Domain Names — Implementation and Specification," RFC 1035, 1987.
- [8] A. Hubert and R. van Mook, "Measures for Making DNS More Resilient against Forged Answers," RFC 5452, 2009.
- [9] J. Damas, M. Graff, P. Vixie, "Extension Mechanisms for DNS (EDNS(0))," RFC 6891, 2013.